

<< 算法设计与分析 >>

§1. Basis

1. 算法

① 解决特定问题的有限指令序列

② 有穷性、确定性、可行性、输入、输出

e.g. 欧几里得算法

$\text{Euclid}(m, n)$: // 输入: 非负整数 m, n , 输出: $\text{gcd}(m, n)$

while $n > 0$ do

$r \leftarrow m \% n$

$m \leftarrow n$

$n \leftarrow r$

return m

! 原理: $\text{gcd}(m, n) = \text{gcd}(n, m \% n)$

① 证明算法正确性:

循环不变量: 指在 loop 每次迭代前后均为真的一性质

if: set 不变量: $\text{gcd}(m_i, n_i) = \text{gcd}(m_0, n_0)$

初始化: true

保持:

数学归纳法: $\text{gcd}(m_i, n_i)$

$= \text{gcd}(n_i, m_i \% n_i)$

$= \text{gcd}(m_{i+1}, n_{i+1})$

终止:

$\Rightarrow \text{gcd}(m_0, n_0) = \text{gcd}(m_k, n_k)$ // 共执行 k 次迭代

$= \text{gcd}(n_k, 0) = n_k$

初始化: before loop, true

保持: 每次迭代后, true

终止: false $\rightarrow$ 推出结果正确

② 运行效率: 时间复杂度

$\rightarrow$ 屏蔽层的基本运算次数

No.

DATE / /

§ 2. 渐近分析.

? Why need 渐近分析? 精确计算时间复杂度 $T(n)$ 十分复杂.

and in fact, we only care $n \rightarrow \infty$ 时的时间增长率.

△ 5个渐符号.

符号	定义	数学定义.
O	上界	$\exists c, n_0 > 0$ s.t. $\forall n > n_0, 0 \leq f(n) \leq c \cdot g(n)$. 记作 $f(n) = O(g(n))$.
o	非紧确上界	$\forall c, \exists n_0$ s.t. $\forall n > n_0, 0 \leq f(n) < c \cdot g(n)$
Θ	紧确界	$f(n) = c \cdot g(n)$
ω	非紧确下界	$0 \leq c \cdot g(n) < f(n)$
Ω	下界	$0 \leq c \cdot g(n) \leq f(n)$

△ Common 复杂度.

$$1 \ll \log \log n \ll \log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n! \ll n^n$$

Stirling 公式: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \Rightarrow \log(n!) = \Theta(n \log n)$ (pl CS中, $\log$ 底默认为 2.)

调和级数: $\sum_{k=1}^n \frac{1}{k} = \Theta(\log n)$

$$n! = o(n^n)$$

$$n! = \omega(2^n)$$

$$\Delta f(n) = o(g(n)) \iff f(n) = O(g(n))$$

e.g. $f(n) = n^2 \neq n$

$$f(n) = O(n^2), f(n) = O(n^3), f(n) = o(n^3), f(n) \neq o(n^2)$$

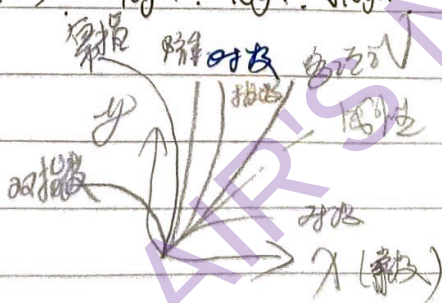
△ 排序: $2^2, n^n, n!, n \cdot 2^n, (3/2)^n, (\log n)^{\log n} = n^{\log \log n}, n^3$
 (DESC) 双指数, 指数, 阶乘, 指数, 指数, 指数, 多项式.
 $\downarrow$
 $2^{n \log n}$

$$\log(n!) = \Theta(n \log n), n = \Theta(2^{\log n})$$

$$\log \log n, n^{\frac{1}{\log n}} = \Theta(1)$$

双对数.

$$(2^{\log n})^{\frac{1}{\log n}} = 2.$$



§3. 递归.

引例: Hanoi Tower. $T(n) = 2T(n-1) + 1$. 且 $T(1) = 1$.

$$\Rightarrow T(n) = 2^n - 1 \quad \text{推导} \quad \text{迭代法: } T(n) = 2T(n-1) + 1$$

1. 解递归方程的方法.

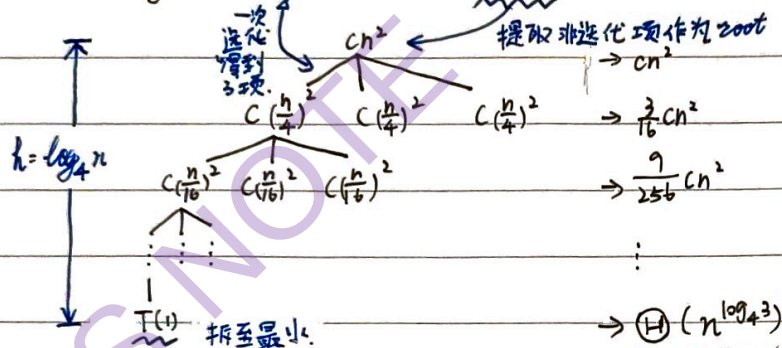
A. 迭代归纳法.

e.g. $W(n) = W(n-1) + n$ (直线划分平面
每加一条线
最多加 n 个区域).

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 4T(n-2) + 2 + 1 \\ &= 8T(n-3) + 4 + 2 + 1 \\ &\Rightarrow T(n) = 2^k T(n-k) + \sum_{k=0}^{n-1} 2^k \\ &= 2^{n-1} T(1) + \frac{1 \cdot (1-2^n)}{1-2} \\ &= 2^n - 1. \end{aligned}$$

B. 递归树法.

e.g. $T(n) = 3T(n/4) + \Theta(n^2)$.



$$\begin{aligned} L_1 &: 3 \times \left(\frac{1}{4}\right)^2 \times cn^2 \\ L_2 &: 3^2 \times \left(\frac{1}{4}\right)^2 \times cn^2 \\ L_3 &: 3^3 \times \left(\frac{1}{4}\right)^2 \times cn^2 \\ &\vdots \\ L_n &: 3^n \times \left(\frac{1}{4}\right)^2 \times cn^2 \\ &= \left(\frac{3}{16}\right)^n \cdot cn^2 \\ L_h &= \left(\frac{3}{16}\right)^{\log_4 n} \cdot cn^2 \\ &= \frac{3^{\log_4 n}}{16^{\log_4 n}} \cdot cn^2 = \frac{3^{\log_4 n}}{n^2} \cdot cn^2 \\ &= c \cdot 3^{\log_4 n} = \Theta(3^{\log_4 n}). \\ &\text{化简: } 3^{\log_4 n} = 3^{\frac{\log_2 n}{2}} = n^{\frac{\log_2 3}{2}} = n^{\log_4 3} \\ &\Rightarrow \Theta(n^{\log_4 3}). \end{aligned}$$

$$\begin{aligned} \Rightarrow T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i \cdot cn^2 + \Theta(n^{\log_4 3}) \\ &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i \cdot cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\ &= O(n^2). \end{aligned}$$

验证: $T(n) \leq 3 \cdot d \cdot L_n^2 + cn^2$ 假设 $T(n)$ 满足.

$$\leq 3d \cdot \left(\frac{n}{4}\right)^2 + cn^2$$

$$\leq dn^2$$

当 $d \geq \frac{16}{3}c$ 时, 成立.

C. 主定理.

层数 $k \approx \log_b n$.

$$T(n) = aT(n/b) + f(n) \quad (a \geq 1, b > 1) \quad \text{递归子问题的增长率: } n^{\log_b a}$$

比较结果.

复杂度

理解.

$$f(n) = O(n^{\log_b a - \epsilon})$$

$$T(n) = \Theta(n^{\log_b a})$$

叶子节点 (递归部分) 太重, 占主导.

$$f(n) = \Theta(n^{\log_b a})$$

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

递归 & 非递归部分均力敌.

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

$$T(n) = \Theta(f(n)). \quad \text{层数.}$$

根节点占主导. (需满足正则条件 $\exists c, 1 \leq n$
下层 $\leq$ 层 $\leq af(n/b) \leq cf(n)$)

No.

DATE / /

 $\Delta \varepsilon$ 的作用递归部分的阶: $n^{\log_b a}$ 假设 $f(n)$ 的阶: n^k 为判断二者是否有阶的差距, 如情况三, 要求 $k > \log_b a$ 量化 $\rightarrow k = \log_b a + \varepsilon$ Δ 定理条件间的空隙

e.g. $T(n) = 2T(n/2) + n \log n$

$f(n) = n \log n$

 $n^{\log_b a} = n \Rightarrow$ 并未大出一个多项式因子 n^ε , 只大了一个对数因子 $\log n$ $\therefore$ 定理失效

e.g. 求递归方程: $T(n) = 3T(n/4) + n \log n$

$\checkmark \varphi: \text{取 } a=3, b=4, f(n) = n \log n$

$\therefore n^{\log_b a} = n^{\log_4 3}$

$\therefore f(n) = n \log n = O(n^{\log_4 3 + \varepsilon}) \quad (\varepsilon = 0.2)$

要使 $a f(n/b) \leq c f(n)$ 成立

$\checkmark \quad 3 f(n/4) \leq c f(n)$

$\checkmark \quad 3 \cdot \frac{n}{4} \log \frac{n}{4} \leq c \cdot n \log n$

显然只要 $c > \frac{3}{4}$ 即可满足对充分大的 n 成立

这里相当于定理的第三种情况

$\therefore T(n) = O(n \log n)$

e.g. $\begin{cases} T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + n \\ T(1) = 1 \end{cases}$

$\checkmark \varphi: \because T(n) = 2T(\frac{n}{2}) + n$ 可证为 $O(n \log n)$

$\checkmark \quad \exists c > 0, n_0 \text{ s.t. } \forall n \geq n_0, \text{ 有 } T(n) \leq c n \log n$

可估计原递归方程的解也是 $c n \log n \quad (n \geq 2)$

$\checkmark \text{归纳证明: 当 } n=2, T(2) = 2T(1) + 2 = 4, \text{ 要使 } 4 \leq 2c \log 2, c \text{ 可取 } 2$

$\checkmark \quad T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + n \leq 2T(\frac{n}{2}) + n \leq 2c \frac{n}{2} \log \frac{n}{2} + n$

$\downarrow = c n \log n - c n + n \leq c n \log n \quad (c=2)$

 $\frac{n}{2}$ 成立 $\Rightarrow n$ 成立
 $\downarrow$
一切成立

§4. 分治策略 <上>

核心思想: *Divide* $\rightarrow$ *Conquer* $\rightarrow$ *Merge*

时间复杂度: $W(n) = W(|P_1|) + W(|P_2|) + \dots + W(|P_k|) + f(n)$

非递归代价.
常通过均匀划分以降低.

案例一. 数值计算优化.

e.g. A. 幂乘运算 a^n .

$$\text{分治: } a^n = \begin{cases} a^{n/2} \times a^{n/2} & (\text{偶}) \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & (\text{奇}) \end{cases}$$

$$W(n) = W(n/2) + \Theta(1) \Rightarrow \Theta(\log n)$$

e.g. B. Fibonacci

$$\text{M. 定理: } \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

实数幂运算
推广至矩阵.

$$W(n) = \Theta(\log n)$$

案例二. 最大子数组.

法A: 暴力 (*Brute Force*). $O(n^2)$.

法B: 分治. $\rightarrow$ 从中间将数组二分. 最大子数组落点.

Left part
Cross midpoint
Right part

递归找两侧 max.

针对 cross 情况, 从中间向两侧扫描, 找 max 前缀 & 后缀, 求和. $\Theta(n)$.
return 三者 max.

$$T(n) = 2T(n/2) + \Theta(n)$$

根据主定理. 类型二, $T(n) = \Theta(n \log n)$.

§5 分治策略 <下>

1. 改进分治算法的路径:

$$T(n) = aT(n/b) + f(n) \Rightarrow \begin{matrix} \text{子问题数} \\ a \downarrow \end{matrix} \quad \begin{matrix} \text{子问题规模} \\ n/b \downarrow \end{matrix} \quad \begin{matrix} \text{非递归部分} \\ f(n) \downarrow \end{matrix}$$

A: 减少子问题个数 a : e.g. Karatsuba 乘法、Strassen 矩阵乘法.

B: 降低合并代价 $f(n)$: e.g. 平面最近点对问题

2. 案例

Case 1: 大整数乘法: calculate 2个 n 位大整数 X 与 Y 的积.

a. 普通竖式乘法: $O(n^2)$.

b. 朴素分治: $XY = (10^{n/2}A + B)(10^{n/2}C + D)$.

$$= 10^n AC + 10^{n/2}(AD + BC) + BD.$$

$$\therefore T(n) = 4T(n/2) + O(n)$$

$$\Rightarrow T(n) = O(n^2).$$

c. Karatsuba 算法:

$$AD + BC = (A+B)(C+D) - AC - BD$$

$$T(n) = 3T(n/2) + O(n)$$

$$\Rightarrow T(n) = O(n^{\log_2 3})$$

Case 2: 矩阵乘法 (方阵).

a. 直接计算: $O(n^3)$.

b. 朴素分治: 将矩阵拆分为 4 个分块矩阵.

$$A \cdot B = C \Rightarrow \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$$\therefore T(n) = 8T(n/2) + O(n^2)$$

$$\Rightarrow T(n) = O(n^3).$$

c. Strassen 算法: 通过 7 个中间矩阵, 使得: $T(n) = 7T(n/2) + O(n^2)$.

$$\Rightarrow T(n) = O(n^{\log_2 7}).$$

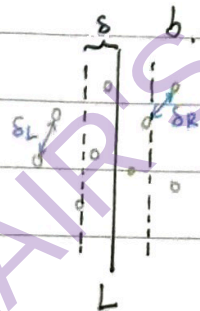
Case 3: 平面最近点对.

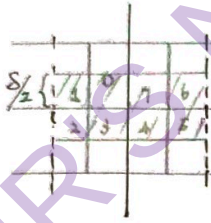
a. 暴力: $O(n^2)$.

b. 分治: ①分: 用垂直线 L 将点集分为 S_L 和 S_R .

②治: 递归求出两部分内最近距离, $\delta = \min(\delta_L, \delta_R)$.

③合: 检查分居两侧的点 (对 P_L, P_R). $\rightarrow$ 只需检查 L 两侧 δ 区域 (条状区域).





几何性质: 针对条形区域内每个点, 只需 check 其下方点 (理论 max 7 个)
(如均在上方也行).

explain: 1. 限制范围 < 横轴: 28.
纵轴: 8.

2. The Grid: (如左图), 切成 8 份.

3. each 格子最多 1 个点.

4. 理论上只需考虑另侧的 4 个, but 为便于 coding, 要 check 7 个.

Now, we possess one array Y ordered by y.

So, it's convenient to check the following 7 points without caring about side.

version 1: 每次合并时对区域内点按 y 排序.

$$T(n) = 2T(n/2) + O(n \log n) \quad \text{根据递归树 (理论 X)}$$

$$\Rightarrow T(n) = O(n \log^2 n)$$

version 2 (改进): 将排序工作移出递归.

before 递归, 生成 < 按 x 排序的 P_x .
... P_y .

No.

DATE / /

§6. 动态规划 <上>

1. Basis

动态规划 = 递归 + 记忆

分解子问题. 消除重复计算.

最优子结构

重叠子问题. (空间换时间)

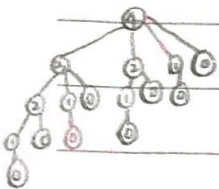
2. 案例

Case 1. 钢条切割: 给定长度为 n 的钢条, 和价格表 P , 求最大收益 r_n .

a. 暴力搜索: 割法有 2^{n-1} 个. (可行解)

b. DP.

(最优性)



$$r_n = \max_{1 \leq i \leq n} (P_i + r_{n-i}) \quad // \text{ 设最优解有切割, 则可穷举第一刀可能.}$$

⇒ 伪代码: CUT-ROD(P, n)

if $n == 0$

return 0

$q = -\infty$ // 收益

for $i = 1 \rightarrow n$

$q = \max(q, P[i] + \text{CUT-ROD}(P, n-i))$

return q

大量重复.

$$\Rightarrow T(n) = 1 + \sum_{j=0}^{n-1} T(j) = 2^n$$

↓ 改进 A & B.

改进: A. 自顶向下备忘 (Top-down with memorization)

递归 + 记忆 memo, 每次计算前先查 memo 是否存值.

⇒ pseudocode: ROD-CUT(P, n, r)

if $r[n] > 0$ return $r[n]$ // 算过即直接引用.

if $n == 0$ $q = 0$

else

$q = -\infty$

for $i = 1 \rightarrow n$

$q = \max(q, P[i] + \text{ROD-CUT}(P, n-i, r))$

$r[n] = q$

return q

⇒ 复杂度为: 平方级.

pl. 递归 (Top-down)

↓ 空间换时间.
DP

B. 自底向上 (Bottom-up). (推荐)

计算 $r_i \rightarrow r_n$, 当计算 r_j 时, $r_1 \rightarrow r_{j-1}$ 已算好. 可查表.

⇒ pseudocode: ROD-CUT(P, n)

if $q < P[i] + r[j-i]$ // 最优值 → 最优性.
 $q = P[i] + r[j-i]$ // 记录切割方案.
 $S[j] = i$ // 记录切割点.

⇒ 复杂度: 平方级.

$r[0] = 0$

for $j = 1 \rightarrow n$

$q = -\infty$

for $i = 1 \rightarrow j$

$q = \max(q, P[i] + r[j-i])$

$r[j] = q$

return $r[n]$

< 3. 一定最优

Case 2. 矩阵连乘: $A_1 \times A_2 \times A_3 \times \dots \times A_n$. (A_i 为二维矩阵, 不一定为方阵.)

满足结合律, 不同运算次序代价差异大.

$$A(BC) = (AB)C$$

P : 存储矩阵维度的数组

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k+1, j] + p_i \cdot p_k \cdot p_j)$$

$P[i-1]$: A_i 行数

$P[i]$: A_i 列数
即 A_{i+1} 行数

计算 $A_i \dots A_j$ 的最小代价.

$\Rightarrow$ pseudocode: MATRIX-CHAIN-ORDER(p)

$n = p.length - 1$ // 矩阵数 n .

定义 m & s // m : $A_i \dots A_j$ 的最小代价; s : $A_i \dots A_j$ 的最优分割点.

for $i = 1 \rightarrow n$ $m[i, i] = 0$

for $l = 2 \rightarrow n$ // 连乘矩阵的个数, 先算出 all $l=2$ 的矩阵连乘, 再算 $3, \dots, n$.

for $i = 1 \rightarrow n - l + 1$ // 连乘起点

$j = i + l - 1$ // 连乘终点

$m[i, j] = \infty$

for $k = i \rightarrow j - 1$ // 扫描范围内 all 子问题 下尝试分割点 k .

$$q = m[i, k] + m[k+1, j] + p_i \cdot p_k \cdot p_j$$

if $q < m[i, j]$

$$m[i, j] = q$$

$$s[i, j] = k$$

return m & s .

$$\Rightarrow O(n^3)$$

§ 7. 动态规划 <下>

1. 解题流程:

① 刻画最优子结构: 证明原问题最优解, 包含子问题最优解.

② 递归定义最优值: 写状态转移方程, (注意边界条件).

③ 计算最优值: Bottom-up

④ 构造最优解: 利用辅助数组回溯.

2. 案例:

Case 1. 最长公共子序列 (LCS)

给定: $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$.① 暴力: 又有 2^m 个子序列. 每次检查 $O(n) \Rightarrow O(n \cdot 2^m)$.② DP: 关注其中的选择: whether to select the last one of X & Y .

$$C[i, j] = \begin{cases} 0 & i=0 \text{ or } j=0 \\ C[i-1, j-1] + 1 & x_i = y_j \\ \max\{C[i-1, j], C[i, j-1]\} & x_i \neq y_j \end{cases}$$

$\swarrow$
 X 的前 i 个和 Y 的前 j 个
 字符的 LCS 长度.

$\searrow$
 若 $x_i = y_j$, 则最后两字符同
 则 $LCS+1$, 包含这个字符.
 else, $x_i \neq y_j = 1$.

pseudo code:

LCS(X, Y): $m = X.length; n = Y.length;$ // 长度.for $i \leftarrow 0$ to m do $C[i, 0] = 0$ > when $i=0$ or $j=0$ $C=0$ for $i \leftarrow 0$ to n do $C[0, i] = 0$ for $i \leftarrow 1$ to m do// $i: 1 \rightarrow m$ for $j \leftarrow 1$ to n do// $j: 1 \rightarrow n$ if $X[i] == Y[j]$ then $C[i, j] = C[i-1, j-1] + 1$ $B[i, j] = '\swarrow'$ else if $C[i-1, j] > C[i, j-1]$ then $C[i, j] = C[i-1, j]$ $B[i, j] = '\uparrow'$

else

then $C[i, j] = C[i, j-1]$ $B[i, j] = '\leftarrow'$

> 遍历: 从小到大. (bottom-up)

Structure Sequence (B, i, j):

if $i=0$ or $j=0$ then return;

if $B[i, j] = '\leftarrow'$

then output (Z[i])

Structure Sequence (B, i-1, j-1)

else if $B[i, j] = '\uparrow'$

then Structure Sequence (B, i-1, j)

else // ' $\rightarrow$ '

then Structure Sequence (B, i, j-1)

Case 2 0-1 背包问题.

给定: n 个物品, 重量 w_i , 价值 v_i , 背包容量 b , 求最大价值.

选择: whether to put in the k -th item.

$$F_{k,y} = \max \{ F_{k-1,y}, F_{k-1,y-w_k} + v_k \}$$

← 前 $k-1$ items, 重量 $\leq y$ 的最大价值

→ 不选 k -th

→ 选

时间复杂度 = 表格格数 $\times$ 表格化简时间 = $(n \times b) \times O(1) \Rightarrow O(nb)$

空间 $\dots: O(nb)$

→ $F_{k,y}, k \in [0 \dots n], y \in [0 \dots b]$.

<伪多项式时间 算法>

Case 3 最大子数组和.

→ 连续.

给定: 数组 $A[1..n]$, 找子数组 $A[i..j]$, 使和最大.

① 暴力: $O(n^2)$

② 分治: $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$

③ DP:

$$C[i] = \max \{ C[i-1] + A[i], A[i] \}$$

$$\Rightarrow \text{OPT}(A) = \max \{ C[i] \}$$

$$T(n) = O(n).$$

§8. 贪心法 <上>

1. 案例: 活动选择问题: 找最大互不冲突子集.

◦ 贪心策略: 结束时间早优先.

◦ 关键要素:

(1) 贪心选择性质: 做出局部最优选择, 一定能导致全局最优.

证: (剪切-粘贴法).

假设一个最优解 A_k . 将第一个活动非贪心选择的 a_m , 则用 a_m 替换第一个 a_j .

依然可行: a_m 比 a_j 结束早, so 不会与 others 冲突.

依然最优: 集合大小不变.

$\therefore$ 贪心选择安全.

(2) 最优子结构: 贪心选择后, 子问题最优 + 已选 = 原问题最优.

2. 一般步骤.

◦ 思路: 通过贪心选择改进最优子结构, s.t. 选后仅剩一个子问题.

① 将优化问题转化 $\rightarrow$ 选后剩单个子问题.

② 安全: 满足性质.

③ 满足最优子结构.

§9. 贪心法 <下>

No.

DATE / /

1. 典型应用:

Case 1: Huffman 编码. 解决无损压缩, 得到最优前缀码.

贪心选择: 找权值最小的 2 棵树合并.

正确性证明:

① 引理 1 (性质): 频率 $\min$ 的 2 字符 x, y , 在最优树中为兄弟节点, 且在最底层.

证: 设 x, y 非兄弟/兄弟, 则可进行 swap, 后带路径长度 WPL 将变小或不变, 不会增.

② 最优子结构: $f(x) + f(y) = f(z)$, 原优 = 子优 + $f(z)$.

? 合并后的子问题最优解 T' , 那么还原回去的解 T 为原优吗?

证: (反证法).

C : 原符号集, C' : 合并后符号集.

反设: T 非原优, 故存在 T' 为原优, $WPL(T') < WPL(T)$.

由引理 1, 可合并 T' 得到 T'' (x, y 为 $\min$)

故 $WPL(T'') + (f(x) + f(y)) < WPL(T') + (f(x) + f(y))$

$WPL(T'') < WPL(T')$

但已知 T' 为子优, 不会有 T'' 比 T' 小, 故矛盾.

$\therefore T$ 为原优.

Case 2. 最小生成树.

原图 $G = (V, E)$: n 顶点, m 条边.

A. Prim 算法: 初始 $\{v_0\}$, 选邻接权最小的边加入 (边的另一端点为已连接点集 U , 另一端点为 $V-U$ 中).

正确性证明: 反证 + 剪枝-粘贴.

设最优树不含要加入的最短边 e , 则加 e 会含回路.

而回路中 $\exists e' > e$, 则可将 e' 用 e 替换, 结果更优.

B. Kruskal 算法: 不断 add 最短且不会成回路的边, 类似.

No.

DATE / /

§ 10. 回溯.

1. 回溯法:

- BC: 一种遵照某种规则 (avoid 遗漏)、跳跃式 (剪枝) 地搜索解空间的技术.
- Core: DFS + 剪枝.
- 适用性: 搜索问题 (找可行性) & 优化问题 (找最优解).

2. 关键建模要素:

① 解: $\langle x_1, x_2, \dots, x_n \rangle$

② 解空间: the set of all candidate solutions, 通常树形表示.

③ 约束条件: 剪枝依据.

④ 目标函数: ^{constraint} 评估解的质量.

3. 案例.

Case 1: N皇后

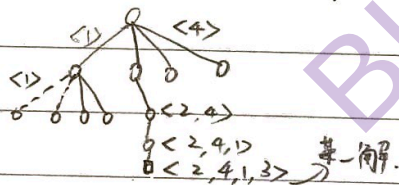
① 解: $\langle x_1, x_2, \dots, x_n \rangle$. x_i 是 i -皇后放置皇后所在的列号.

② 解空间: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid 1 \leq x_i \leq n, i=1, \dots, n \}$

③ 约束: 列: $x_i \neq x_j, i \neq j$.

斜线: $|x_i - x_j| \neq |i - j|, i \neq j$ (巧妙啊!)

e.g. 4皇后 $\rightarrow$ 构造4叉树 $\Rightarrow$ N皇后: N叉树.



最坏情况: $O(n^{n+1})$.

Case 2: 0-1 背包

① 解: $\langle x_1, x_2, \dots, x_n \rangle$. $x_i = 1$ 表示 i -th 物品被选中.

② 解空间: $\{0, 1\}^n$

③ 约束: $\sum w_i \cdot x_i \leq b \rightarrow$ 超重则剪枝.

④ 目标函数: $\max \{ \sum v_i \cdot x_i \}$

最坏: $O(2^n)$.

Case 3: 旅行商 (TSP): 经过 all cities 并回到起点的 Hamiltonian circuit.

① 解: $\langle l_1, l_2, \dots, l_n \rangle$ 为路线. 设 $l_1 = 1$.

② 解 space: 以 1 开头的 $1 \sim n$ 的排列.

③ no constraints

④ 目标函数: $\min \{ \sum l(i_j, i_{j+1}) + l(i_n, i_1) \}$

复杂度: $O((n-1)!)$.

4. 回溯法适用条件.

△ 多米诺性质: 若部分解 $P(x_1, \dots, x_k)$ 不满足约束.

Dominance property.

then all 扩展解 $P(x_1, \dots, x_{k+1})$ 均不满足.

↓
若不满足多米诺性质, 须先进行变量变换使其满足.

e.g. $5x_1 + 4x_2 + x_3 \leq 10, 1 \leq x_k \leq 3$.

令 $x_3' = 3 - x_3, 0 \leq x_3' \leq 2$

$\Rightarrow 5x_1 + 4x_2 + x_3' \leq 13$

5. 蒙特卡洛 (Monte Carlo) $\rightarrow$ 估算树的结点数, 即实际搜索空间 $\rightarrow$ 效率估计.

△ 方法: 从根结点, 采用随机采样对 x_i 赋值, 直至得到一完整路径.

利用结果 $\rightarrow$ 无偏估计量 N .

重复采样, 取 AVG, 根据大数定理, 得到估计值.

△ 计算: 某结点有 k 个分支, 随机选 j 个分支, 得到估计值 N_j .

$\Rightarrow$ 树结点估计值: $k \times N_j + 1$.

§ 11. 分支限界.

1. 分支限界:

- AO: 用于求解组合优化问题的算法.
- 机制: Based on 回溯, during DFS, 引入额外约束(界)来剪枝.
- Core: 如果一个部分性无论如何扩张, 都不能 surpass 已有 OPT, 则剪枝.
- Concepts:

• 界: 已找到最优解中的目标函数值.

Bound

• 代价函数: 对于树最优结果的估计.

• 判定法则: (max问题) 如果当前 node 的估算上界 $UB <$ 已知最大值 LB , 则剪枝.

2. 案例:

Case 1: 0-1 背包.

① preprocess: sort by (v_i/w_i) DESC.

单位重量价值.

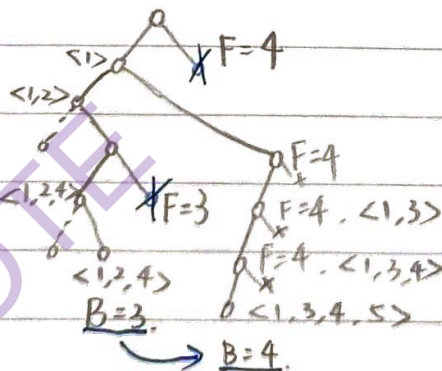
② 代价函数 (VB):

$VB =$ 已装入价值 + 剩余空间 $\times$ next item 单位价值.

Case 2: 最大团问题: 求顶点数最多的完全子图.

$VB =$ 当前团内顶点数 + 剩余顶点数.

e.g. 解: $\langle v_1, v_2, v_3, v_4, v_5 \rangle$ 中 v_i 表示 i 号顶点是否入团.
枚举: (左为 1, 右为 0: 不入)



F: cost function.

B: bound.

Case 3. TSP. Travelling salesman problem.

type: minimize.

① $LB = \text{当前路径长度} + \text{最短出边} + \text{剩余顶点最短出边和}$

压缩搜索空间

$$B = \{1, i_1, \dots, i_k\}$$

$$\min_{l \neq i_k} \{i_k, l\}$$

$$\sum_{j \in B, l \neq j} \min \{j, l\}$$

② $LB = L_{\text{curr}} + i_k \text{ 最短出边} + 1 \text{ 最短出边} + \text{MST}\{V-B\}$

$$\min_{l \in B} \{i_k, l\}$$

$$\min_{l \in B} \{1, l\}$$

$$V-B \text{ 的最小生成树}$$



§12. 问题复杂性.

1. Basis

△问题复杂度：一个问题的最高效解决算法的复杂度

↳ 复杂度④ $f(n) <$ 可达性: 复杂度为 $O(f(n))$ 的算法.

下界: $\forall$ 算法复杂度 $\Omega(f(n))$.

算法上界 O 与问题下界 Ω 重合 $\Rightarrow$ 渐进最优算法.

2. 确定下界的方法.

① 执行下界: 根据输入流变量及输出流变量确定

e.g. 1. 写出 all n 阶置换 $\xrightarrow{n! \text{ 个解}} \Omega(n!)$

e.g. 2. 多项式求值 $\xrightarrow{n \text{ 个输入数据}} \Omega(n)$

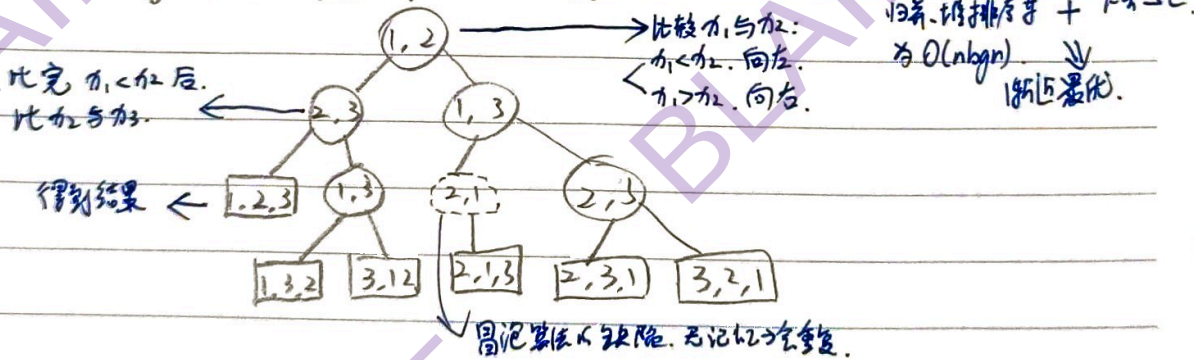
e.g. 3. 矩阵乘法 $\xrightarrow{\text{输入 } n \times n \text{ 个元素}} \Omega(n^2)$

② 决策树: 以比较为基本运算.

• 模型: 二叉树, 内部节点为一次比较, 叶节点为结果

定理: 任何基于比较的排序算法在最坏情况下需比较 $\lceil \log_2 n! \rceil \approx n \lg n$ 次。

e.g. 冒泡: 输入 $\{x_1, x_2, x_3\}$.



0. 定理证明: 最坏比较次数为树高 n .

叶结点权 (结果权): $n!$ (A_n^n).

树高为 n 的叶结点最多: 2^{n-1} (二叉完全树)

$$\Rightarrow 2^h \geq n!$$

$$h \geq \log_2 n! \approx n \log h.$$

③ 对于递归: 为算法构造最好输入.

e.g. 1. Find Max.

o 下界: $(n-1)$ 次比较.

o 理由: n 个数中取 max, 需淘汰 $(n-1)$ 个, 每次比较最多淘汰 1 个.

e.g. 2. Find Max & Min.

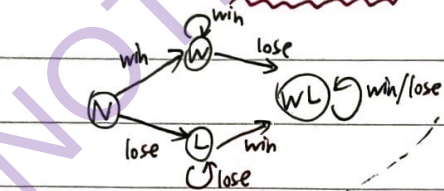
o 下界: $(\lceil \frac{3}{2}n \rceil - 2)$ 次比较.

o 最优策略: ① 两两比较, 分 Win 组 & Lose 组. $(\frac{n}{2})$

② Win 中找 Max, Lose 中找 Min. $(\frac{n}{2}-1) \times 2$

$\Rightarrow \text{total} \approx \frac{3}{2}n$.

o 思路: (状态机分析)



信息单位:

N: 0

W/L: 1

WL: 2

$\Rightarrow \text{start: } 0$

$\text{end: } 2n-2$ (must meet).

ii 得分方式:

$N \text{ vs } N + 2$
 $N \text{ vs } W/L + 1$
 $W \text{ vs } W/L \text{ vs } L + 1$
 $WL \text{ vs } WL + 0$

iii 计算极限: $(2n-2)/2 = n-1$ 次.

$\rightarrow$ 理想情况下, 每次 +2.

iv $(\frac{n}{2}) \times 2 = n$ 分.

实际最多 $\frac{n}{2}$ 次 +2

$[(2n-2) - n] / 1 = n-2$ 次

$\rightarrow$ 每次 +1.

$\Rightarrow \frac{n}{2} + (n-2) = \frac{3}{2}n - 2$ 次.

④ 规约: 利用已知问题的难度, 证明新问题的难度.

问题: $Q \xrightarrow{\text{reduction}} P$

实例: $I \longleftrightarrow A(I)$.

下界: $\Omega(f(n)) \rightarrow \text{至少 } \Omega(f(n))$.

已知:

o 推导流程: 1. Q hard.

2. $Q \xrightarrow{\text{reduction}} P \xrightarrow{P \text{ 的解}} P \text{ 的解} \rightarrow Q \text{ 的解}.$

3. if P easy $\rightarrow Q$ easy. 矛盾

$\therefore P$ also hard.

No.

DATE / /

因子分解 $\text{factor}(n)$. P
e.g. 1. 素数测试 $\text{testp}(n)$. Q
假设 Q 复杂度 $W(n)$.

Q 算法: 1. $p \leftarrow \text{factor}(n)$
2. if p then return false
3. else return true

B // 写完作业后, 就知道了有无作业.
A
 $\Rightarrow T_A \leq T_B$. (in fact, 先完成 A, 再完成 B.)

$$\Rightarrow \Omega(W(n)) = T_Q \leq T_P.$$

e.g. 2. 已知: 元素唯一性问题 Q: 集合 S 中有无重复元素. $W(n) = \Omega(n \log n)$.

问题: 最邻近点对 P

证: 变换 f : Q 的实例: $x_1, x_2, \dots, x_n \xrightarrow{f} (x_1, 0), (x_2, 0), \dots, (x_n, 0)$

P: 1. 用 P 算法解 Q : 求最短距离 d .

2. if $d=0$ then return false

3. else return true

$\Rightarrow$ 若 P 能更快解决, 那 Q 也能, 与 Ω 矛盾. $\therefore$ 不能 $\Rightarrow T_P = \Omega(n \log n)$

§13. NP完全性.

No.

DATE / /

1. Basis.

判定问题

P类: 多项式时间可解.

NP类: 多项式时间可验证.
nondeterministic polynomial

$P \subseteq NP$.

$NP \subseteq NP\text{-hard}$.

多项式时间归约: $A \leq_p B$. 表示可归约到B. $\exists$ 多项式时间算法 f 使 $A(x) \text{ 实例 } I \rightarrow B(f(I))$

具有传递性

$I \in \text{Yes}(A) \Leftrightarrow f(I) \in \text{Yes}(B)$.

e.g. $HC \leq_p TSP$.

证: 对HC的任一实例 $I: G = \langle V, E \rangle$, TSP对应的实例 $f(I)$ 为:

城市集 V , $d(u, v) = \begin{cases} 1, & \text{if } (u, v) \in E \\ 2, & \text{else} \end{cases}$, 界限 $D = |V|$.

$\searrow$ u, v 不同. TSP问题中: 解决的是哈密图. 哈密图 $\Leftrightarrow$ 存在哈密回路. 哈密图 $\Leftrightarrow$ 存在哈密回路. 哈密图 $\Leftrightarrow$ 存在哈密回路.

$\Rightarrow f$ 可在多项式时间计算

$I \in HC \Leftrightarrow f(I) \in TSP$.

Δ 若 $A \leq_p B$, $B \in P \Rightarrow A \in P$.

证: 设 f 是 $A \rightarrow B$ 的多项式时间归约.

$\because B \in P \therefore \exists B$ 的多项式时间算法 g .

$\Rightarrow A$ 的结果为 $g(f(I))$. 为多项式时间算法

$\therefore$ 得证.

NP完全性 (NPC)

Δ NP-hard: 所有 NP 可 $\leq_p$ NP-hard.

Δ NPC = NP & NP-hard.

GNP

$\in$ NP-hard

不定 GNP

NPC: 难. 能够归约.

NP-hard: 难. 不一定能够归约.

★

Δ 证明是 NPC 的方法: (1). 证明 $\Pi \in \text{NP}$

(2). 找一个已知的 NPC 问题 Π' . 证明 $\Pi' \leq_p \Pi$.

2. 案例.

e.g. 1. 可满足性 (SAT) 问题: 给定一个合取范式 F . 判断是否可满足. (是否存在成真赋值).

特例:

3-SAT

Δ 特例: 若 Π 的所有实例均为 Π' 的实例. 则 Π 为 Π' 的特例.

$\Rightarrow$ 性质:

若 $\Pi' \in P \rightarrow \Pi \in P$

若 $\Pi' \in \text{NP} \rightarrow \Pi \in \text{NP}$

若 $\Pi \in \text{NPC} \rightarrow \Pi' \in \text{NPC}$

(逆命题均不成立).

No.

DATE / /

→ 每个简单析取式由3个文字组成的合取范式。

e.g. 1.1. 证明 3-SAT 是 NPC.

证: step 1. 证 3-SAT $\in$ NP.

$\therefore$ SAT $\in$ NP $\therefore$ 3-SAT $\in$ NP.

step 2. 证 SAT $\leq_p$ 3-SAT.

任给 SAT 的 F, 生成 3-SAT 的 F'.

→ 转换: F 中 $C_j = l_1 \vee l_2 \vee l_3 \vee \dots \vee l_k$

$k=1: C_j = l_1 \vee l_1 \vee l_1$

$k=2/3$ 同理扩充.

$k=4: C_j = (l_1 \vee l_2 \vee \neg) \wedge (l_3 \vee l_4 \vee \neg)$

冗余文字: 若 l_1, l_2 为真, 则为假.
else 为真.
s.t. 成立.

→ $C_j = (l_1 \vee l_2 \vee \neg) \wedge (\neg l_1 \vee l_3 \vee \neg) \wedge \dots \wedge (\neg l_{k-3} \vee l_{k-2} \vee \neg)$

理由: l_i 为真, 则令其他文字为真.

e.g. l_i 为真, 则令 $\neg l_i \rightarrow$ 为真. (前)

$\neg l_i \rightarrow$ 为假. (后)

→ 利用其他文字
不影响合取结果
为真.

$\therefore$ F' 可在多项式时间内生成.

$F \in \text{SAT} \Leftrightarrow F' \in 3\text{-SAT}.$

$\therefore$ SAT $\leq_p$ 3-SAT

综上, 3-SAT 是 NPC.

→ max 完全图.

e.g. 2. 最大团问题 (CLIQUE): 给定 $G=(V, E)$ 和 $k \in \mathbb{N}^+$. 问: G 中是否存在大小为 k 的团?

step 1. 显然, CLIQUE $\in$ NP.

step 2. 证 3-SAT $\leq_p$ CLIQUE.

令 $\Phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ 为 3-SAT 输入. 其中 $C_i = (l_1^i \vee l_2^i \vee l_3^i)$

构造图: 1. 顶点: each C_r 在 G 中构造 3 个顶点 v_1^r, v_2^r, v_3^r .

(共 $3R$ 个顶点, each 对应 Φ 中一个文字位置).

构造顶点 $\rightarrow 3R$

构造边: $C_R^2 \times O(1)$

2. 边: 不同的 $\textcircled{A}: v_i^r$ 与 v_j^s 中, $r \neq s$. (同-构造 3 点间无边).

例: 团-构造 $\textcircled{B}$ 2 文字不互为否定.

$\Rightarrow O(R^2)$.

$\therefore$ 构造是在多项式时间内的.

3. 目标团大小为 k.

正确性证明: 必要性 ($\Rightarrow$): Φ 可满足 $\Rightarrow$ 有目标团.

可在 Φ 每个子句中找一个为真的文字, 共 k 个顶点

→ 满足 $\textcircled{A}$

→ 均为真, 满足 $\textcircled{B}$.

充分性 ($\Leftarrow$): G 有同构 $\Rightarrow$ 可满足

由于同一子句顶点互不相连, k 个顶点属于 k 个子句中, 各子句中各取一个.

真值赋值: 将 k 个文字赋真值, 其余作假值.

→ 该字句不会出现逻辑冲突 (无矛盾同时为真)

$\therefore$ 使得 each 子句至少有一个成真文字, 得证.

§14. 难问题处理策略.

△ NP-难是基于最坏情况定义的. 实际应用中, 往往没有那么极端.

△ 处理策略:

穷举

启发

精确求解

高效的启发式算法.^①

简化要求

随机算法.^② 近似算法.^③ 伪多项式时间算法.^④

启发式搜索^⑤

基于经验/局部改进, 无严格质量保证.

注解: ① When n 较小, 代价可接受.

② 用正确率换速度. 过程中有随机选择 (而非遵循固定的确定性规则).

e.g. Monte Carlo 和 Las Vegas.

③ 不求最优值.

④ e.g. 0-1 背包: NPC. 复杂度 $O(nb)$. $\Rightarrow$ 当 b 较小时, 算法高效.

③' 近似率:

$$d_A = \max_I \frac{A(I)}{OPT(I)}.$$

e.g. 最小点覆盖问题 (Min-VCP). NPC.

pt. 点覆盖: cover all edges 的顶点集.

通过“极大匹配”问题求解:

1. 找出一组极大匹配 A (互不相邻的边). pt. 匹配: 一组边中, 任两边无公共端点 (分散).

2. Add all 边的端点 into set C .

3. C 为一个点覆盖. 且为 2 -近似的.

4. 证明思路: 为 cover A 中边, OPT 至少 need $|A|$ 个点;

而 our 算法取了 $2|A|$ 个点. $\Rightarrow$ 误差 ≤ 2 倍.

证明: $\because A$ 极大. 假设 $\exists$ 边 v 未被 C cover.

则其 2 端点都不在 A 中. 那其本可以加入 A .

故矛盾.

⑤ 局部搜索: 定义邻域映射. 只走向比当前解更优的邻居.

当邻域内无更优解. stop & output local optima.

缺点: 1. 局部最优而不能保证全局最优.

2. 依赖初始值.